STUDENT CAPSTONE BRIEFS

Capstone Projects

Copy question

ON THIS PAGE

1. Submission Guidelines (read first)
2. Your scenario
3. Part 1 — Dataset Design & RAG Core (30 marks)
4. Part 2 — CrewAI Multi-Agent Orchestration with Tools, Memory & Guardrails (30 marks)
5. Part 3 — Evaluation, Observability & FastAPI Deployment (20 marks)
6. Part 4 — Resilience & Governance (20 marks)
7. Acceptance criteria (your submission is complete when...)
8. Submission

Final Capstone — Naukri.com Domain Support Agent (`CrewAI`)

Track: Recruitment & HR (Naukri.com). Estimated duration: 14 days.

Real production support agents combine several building blocks into one working system:

- dataset design
- embeddings
- vector retrieval
- grounded generation
- agent memory
- tool-calling agents
- multi-agent orchestration
- structured outputs
- guardrails
- evaluation
- AI governance
- cost control
- `API` deployment

- This capstone asks you to bring all of it together into ONE working, production-minded agent: a domain support agent that answers policy questions from a knowledge base you write
  - looks up real records from a dataset you design and validate yourself
  - remembers a conversation
  - is guarded against misuse
  - has its own answers reviewed by a second independent agent team before they reach the user
  - operates under an explicit governance policy — orchestrated with `CrewAI`
  - deployed behind a `FastAPI` backend
  - evaluated end to end
- Naukri.com's employer-support team wants an agent that answers hiring-policy questions and checks a specific job application's status, so recruiters and candidates get instant answers instead of waiting on email threads.
- You are building that agent
- And building it to a standard where it could pass a real production governance review, not just a happy-path demo.

Total marks: 100 · **One deliverable:** a single public `GitHub` repository containing your dataset, `RAG` core, `CrewAI` agent crew, `Autogen` review stage, and `FastAPI` deployment.

Submission Guidelines (read first)

- Submit **one public `GitHub` repository link** for the whole project.
  - There is no per-Part submission
  - One repo contains everything.
- State at the top of your `README.md` that you completed the **Naukri.com (Recruitment & HR)** track, and include the exact dataset-design choices you made in Part 1 Task 1 (seed, category/status weights, amount range) so the grader can reproduce your dataset deterministically.
- No images, screenshots, diagrams-to-upload, PDFs, slide decks, presentations, video, or audio are required or accepted anywhere in this project.
  - Every deliverable is code or text inside the repository.
- Nothing in this project requires a paid account or a credit card.
  - Embeddings (`SentenceTransformers`) and the vector index (`ChromaDB`) are free and run locally.
  - The agent's language-model calls (grounded generation, the judge used for evaluation, the `CrewAI` crew, and the `Autogen` review team) default to a deterministic `MOCK_LLM` mode that needs zero `API` keys and zero network access
  - This is the mode your graded transcripts must use.
  - A real LLM `API` may optionally be wired in behind an environment-variable flag, but every acceptance criterion below must be demonstrably satisfied under `MOCK_LLM` alone.
- **Implementing `MOCK_LLM` for a `CrewAI` crew is more involved than a simple flag**
  - Extend `crewai.llms.base_llm.BaseLLM` (`CrewAI` 's own documented extension point for a non- `litellm` LLM) rather than trying to intercept calls externally.
  - Two known pitfalls, both silent (no crash, just wrong output) if you miss them:
    - (1) `CrewAI` 's own built-in `ReAct` system-prompt template literally contains the example text `"Observation: the result of the action"` — a parser that searches the conversation for the string `"Observation:"` to extract a tool's result will match this template text on the very first call, before any tool has run, and silently return placeholder text as the final answer; parse the model's own generated text, not the system-prompt template
    - (2) Do not dispatch a tool call's arguments by matching the tool's *name* (e.g. checking whether `"lookup"` is a substring of the tool name) — a tool literally named `rag_lookup` will be silently misclassified; dispatch off the tool's own declared argument schema instead
- **`CrewAI` 's own telemetry attempts an outbound network call by default** when `crew.kickoff()` runs, which would silently violate the zero-network-access requirement above.
  - Set `CREWAI_DISABLE_TELEMETRY=true` (or `OTEL_SDK_DISABLED=true`) as an environment variable before running your crew, and confirm in your `README.md` that you set it.
- You may refer to official documentation (e.g. the `CrewAI` docs, the `Autogen` docs, the `LangChain` docs at python.langchain.com, the `ChromaDB` docs, the `FastAPI` docs, the Python standard library docs) while writing your code.
- **Originality:**
  - your dataset
  - knowledge base
  - code
  - analysis must be your own work for this specific brief
- Submit your one repository link by the end of the **14-day** capstone window.

Your scenario

**Given category vocabulary (use every value at least once; you may add more):

- ** `Software Engineer`
- `Data Analyst`
- `Product Manager`
- `HR Executive`
- `Sales Associate`

**Given status vocabulary (use every value at least once; you may add more):

- ** `Applied`
- `Screening`
- `Interview Scheduled`
- `Offered`
- `Rejected`

Required knowledge-base topics (≥12 documents, ≥2-5 sentences each):**

- ** job-application eligibility criteria
- interview-scheduling process
- offer-negotiation policy
- background-verification process
- notice-period policy
- referral-bonus policy
- internal-transfer eligibility
- probation-period policy
- remote-work eligibility
- diversity-hiring guidelines
- exit-interview process
- applicant-data-retention policy

- **Guardrail-relevant `PII` fields:** the phone number inside contact details is fixed-format and must be demonstrated masked by your input-side guardrail (Task 10).
- Candidate name, expected salary, and background-check results are free text or unformatted numbers with no reliable pattern to match under a keyless, `MOCK_LLM` -only masker
- Acknowledged as out of scope for masking
- Use only fabricated examples for all three.

Part 1 — Dataset Design & `RAG` Core (30 marks)

Tasks

1. Design and validate your own job-application dataset.

- Write a seeded, deterministic Python generator (`dataset.py`) producing a `JOB_APPLICATIONS` list of **≥40** records.
- Use every category and status value given above at least once (you may add more values).

Each record needs:

- `record_id`
- `category`
- `status`
- `expected_salary_inr` (choose a realistic range and state your reasoning in one sentence)
- `days_since_created` (integer, 0–30)
- `flagged_priority_review` (boolean)

Print and report:

- the count per category (every given category **≥3** records)
- the count per status (every given status **≥1** record)
- the percentage of records with `flagged_priority_review=True` (must land between **10%** and **30%** — if your first random draw doesn't land there, change your seed or weights and regenerate; never hand-edit individual records to force the number)

2. Write your knowledge base.

- Author the **≥12** required documents above (2–5 sentences each, in your own words, covering every required topic).

3. Implement and index two chunking strategies.

- Chunk your documents BOTH as fixed-size-with-overlap AND as sentence-based chunks.
- Embed every chunk in both sets with a free local `SentenceTransformers` model, and index each chunking strategy in its OWN separate `ChromaDB` collection (`collection.upsert()`).

4. Implement grounded generation.

- Given a query, retrieve the top-k chunks (from either collection) and generate an answer using ONLY the retrieved context.
- Under `MOCK_LLM` there is no real model to judge groundedness, so retrieval similarity is your only signal
- **calibrate your "I don't know" threshold empirically before picking one**: measure your own top-1 cosine similarity for at least 3 in-scope queries and at least 2 deliberately out-of-scope queries, then set the threshold between the two clusters you actually observe.
- Do not use an untested preset (0.5/0.6/0.7 are common tutorial defaults that do not reliably separate short policy-sentence embeddings from unrelated queries)
- State your measured values and chosen threshold in `README.md`.
- Demonstrate on **≥5** real in-scope queries plus 1 deliberately out-of-scope query that must trigger the fallback.

5. Evaluate and compare both chunking strategies.

- For the same **≥5** queries from Task 4, compute precision and recall at the document level (map chunks back to parent documents and dedup before scoring) **separately for each of your two collections**, showing per-query arithmetic for both.
- Write 2–3 sentences recommending which strategy you would deploy, citing your own two sets of numbers.

Part 2 — `CrewAI` Multi-Agent Orchestration with Tools, Memory & Guardrails (30 marks)

Using Part 1's `RAG` core (your recommended chunking strategy's collection) as a fixed input, build the orchestration layer.

Tasks

6. Build the second tool with a designed escalation score.

- Using `dataset.py` from Task 1, implement `check_job_application_status(record_id: str) -> dict` returning the application's `status` , `expected_salary_inr` , and a **designed** `escalation_score` in `[0, 1]` that combines `flagged_priority_review` with a normalized recency signal derived from `days_since_created` .
- State your exact formula and the threshold above which you recommend escalation, justified using your own dataset's distribution (e.g. "this threshold corresponds to the 80th percentile of `days_since_created` in my generated data").

7. Build a `CrewAI` crew.

- Construct a crew with **≥3** agents
- At minimum a Retrieval Agent equipped with your `RAG` tool (Task 3–5), a Lookup Agent equipped with your `check_job_application_status` tool (Task 6), and a Response Composer agent that combines their outputs into one final draft answer
- And run it via `.kickoff()` .
- Demonstrate both the `RAG` tool and the lookup tool actually being invoked on different sample queries.

8. Add session memory.

- Using `LangChain` 's session-based memory (e.g. `InMemoryChatMessageHistory` + `RunnableWithMessageHistory` — note: `RunnableWithMessageHistory` raises a `LangChainDeprecationWarning` pointing to `LangGraph` 's own persistence layer; this is expected and does not need to be silenced, the class still functions correctly here), maintain conversation history across turns within one process run and demonstrate state correctly carried across a multi-turn exchange in one transcript, with a **separate** fresh-conversation transcript showing that state correctly absent/reset.
- This memory is in-process only (it does not need to survive a restart)
- That is sufficient for this task.

9. Add a structured output schema.

- Define a `Pydantic` `BaseModel` that every crew response must conform to (`response_format`), and validate each response against it in code.

10. Add guardrails.

- Implement input-side guardrails
- `PII` masking, demonstrated firing on the fixed-format `PII` field(s) identified above, and prompt-injection detection
- And an output-side groundedness check that refuses to answer when the retrieved context doesn't support the question.
- Demonstrate each guardrail actually firing on one deliberate test case.

Part 3 — Evaluation, Observability & `FastAPI` Deployment (20 marks)

Using Part 2's crew as a fixed input, wrap and evaluate it.

Tasks

11. Deploy via FastAPI.

- Expose your crew behind at least 2 `HTTP` endpoints (e.g. `POST /ask` , `POST /add-document`) using `Pydantic` request/response models, **plus** one `WebSocket` endpoint (`@app.websocket`) for real-time multi-turn chat that gracefully handles a client disconnecting mid-conversation (catch `WebSocketDisconnect` , keep the server running for other clients).

12. Add structured logging.

- Log every request as one `JSON-Lines` entry with a trace ID and timing information (ELK-style structured logging).
- The logged request text must NOT contain the raw, unmasked value of any fixed-format `PII` field your Task 10 guardrail masks
- Apply the same masking to what you log as you apply to what the model sees, so a fixed-format `PII` field never reaches disk in the clear.

13. Evaluate with Accuracy, Grounding, Completeness, and Safety at scale.

- Using an LLM-as-judge prompt (running under `MOCK_LLM`), build a test set of **15 queries**
- At least 1 touching every required `KB` topic from your scenario
- Plus at least 2 deliberately out-of-scope or edge-case queries
- And score all four properties for every query
- Reporting all four scores per query **and** the average of each score across all 15.

Part 4 — Resilience & Governance (20 marks)

Using Part 2's crew as a fixed input, harden and govern it.

Tasks

14. Add an `Autogen` review stage after your crew's draft answer.

- Build a 2-agent `Autogen` Round Robin Group Chat team (e.g. a Policy-Compliance-Reviewer agent and a Final-Editor agent) via `RoundRobinGroupChat` , bounded with `max_turns=2` (the real constructor parameter is `max_turns` , not `max_iterations`)
- Or, if you use `MaxMessageTermination` instead, remember it counts the initiating task message as message 1, so `MaxMessageTermination(3)` (not `(2)`) is what actually lets both agents speak.
- Give the Final-Editor a `Pydantic` structured output via `output_content_type=<YourVerdictModel>` on the agent
- This requires the Team itself to also be constructed with `custom_message_types=[StructuredMessage[YourVerdictModel]]` , or the run crashes with `ValueError: Message type ... is not registered` .
- The team takes your `CrewAI` Composer's draft answer plus the original retrieved context as input, and either approves it unchanged or revises it
- The verdict model should have fields `approved: bool` , `final_answer: str` , `reason: str` .
- Demonstrate on **≥2** sample queries: at least one case where the review stage approves the draft unchanged, and at least one case where it actually revises the draft (e.g. catches an ungrounded claim you deliberately injected into the draft for the test).

15. Apply the four-layer AI-governance model.

At the Application layer

- , enforce the principle of least autonomy: only your Lookup Agent may call `check_job_application_status`
- No other agent in your crew may call it directly, and you must demonstrate this restriction (e.g. show that wiring the tool to another agent is blocked or simply never wired, with a one-paragraph explanation of the guard)
- Classify your own system's risk level using the Low/Medium/High risk scheme (Low: summarization/transcription; Medium: code generation/customer support tickets; High: medical data, hiring decisions, financial data), with a one-paragraph justification

At the Runtime layer

- , enforce a per-request token/cost budget cap and demonstrate a deliberately oversized request being rejected rather than silently exceeding the budget

16. Add response caching.

- Implement an in-memory cache keyed by normalized query text for your grounded-generation step.
- Demonstrate a repeated identical query producing a cache hit that avoids a redundant LLM/tool call, with before/after evidence (a call counter or timing comparison).

Acceptance criteria (your submission is complete when...)

Part 1

- `dataset.py` generates **≥40** job applications meeting every stated structural threshold (category counts, status coverage, `flagged_priority_review` percentage band), with your design choices stated in `README.md` .
- Your knowledge base has **≥12** documents covering every required topic.
- Both chunking strategies are implemented, embedded, and indexed into two separate `ChromaDB` collections that both produce sensible retrieval on a sample query.
- Grounded generation is demonstrated on **≥5** in-scope queries plus 1 out-of-scope query that correctly triggers the "I don't know" fallback.
- Precision/recall are computed for BOTH collections on the same **≥5** queries with visible per-query arithmetic, and a numbers-cited recommendation is given.

Part 2

- `check_job_application_status` correctly looks up a record and correctly computes a designed, justified `escalation_score` (not a bare boolean OR).
- The `CrewAI` crew has **≥3** agents (Retrieval, Lookup, Composer at minimum)
  - Both the `RAG` tool and the `check_job_application_status` tool are demonstrably invoked on different queries via `.kickoff()` .
- Multi-turn memory is demonstrated, with a separate fresh-conversation transcript showing it correctly absent.
- Every crew response validates against a declared `Pydantic` structured-output schema.
- Both guardrails (input-side PII/injection, output-side groundedness) are demonstrated actually firing on a deliberate test case each.

Part 3

- The `FastAPI` backend exposes **≥2** working `HTTP` endpoints plus 1 working `WebSocket` endpoint that survives a client disconnect without crashing, all with correct `Pydantic` models where applicable.
- Every request produces one structured `JSON-Lines` log entry with a trace ID.
- All four evaluation scores (Accuracy, Grounding, Completeness, Safety) are reported per-query for all 15 test queries under `MOCK_LLM`
  - Plus the four averages.

Part 4

- The `Autogen` review stage is demonstrated both approving a draft unchanged and revising a draft, with structured `Pydantic` verdicts in both cases.
- Least-autonomy enforcement is demonstrated (only the Lookup Agent can call the lookup tool)
  - A risk classification with justification is given
  - A runtime cost-budget cap correctly rejects an oversized simulated request.
- Response caching is demonstrated producing a real cache hit with before/after evidence on a repeated query.

Submission

- Submit **one public `GitHub` repository link**.
- The repository must contain, in total: `dataset.py` , your knowledge-base documents, your RAG/crew/review-stage/governance/deployment code with transcripts demonstrating every task above, and a `README.md` confirming everything runs under `MOCK_LLM` with zero `API` keys.
- There is exactly one submission link for the whole project.